Assignment 4

Aaron Bussche David Magaram Jason Kasdiran
Group 28

April 14, 2026

Todo list

This is the core conclusion of B.2 — strengthen it with numbers: "On eil51, MA reached mean tour length $X \pm s_X$ vs EA's $Y \pm s_Y$ within the same wall-clock budget of T s (Mann–Whitney U, $p < 0.01$)." Without numbers this reads as an assertion. 7

Note the two paragraphs above (the "We considered. . . " block and the "Comparing the MA. . . " block) are near-duplicates of each other — one is clearly an earlier draft. Keep the better one (the second is more polished) and delete the other. 7

GLOBAL: target grade 8+. Rubric-driven checklist below. Remove todos before submit.

Introduction

Evolutionary algorithms (EA) are a population-based optimization method. They are based on natural selection and work by exploring new solution areas and iteratively refining good already found solutions, where candidate solutions are improved by mutating and crossover. A common issue is the trade-off between exploration and exploitation. Too much exploration can lead to slow convergence, while too much exploitation can cause suboptimal solutions. Despite this, EAs are still very useful, especially for larger, more complex problems. However, their performance depends on parameter configurations such as mutation rate, population size, and selection pressure.

By investigating the traveling salesman problem and string search, this project explores how different parameter configurations influence convergence speed, population diversity, and algorithm performance. It also aims to explore how mutation rate and selection pressure shape the exploration-exploitation trade-off in these settings, and whether a 2-opt local search can outperform a standard EA algorithm on the traveling salesman problem under fair settings.

Maybe add that they can work in discrete spaces/without gradient

Methods

Add a "Reproducibility" paragraph at the end of Methods: language/version (Python X.Y), RNG seeding strategy, how many runs, where the code lives (repo URL or zip), and how to reproduce each figure (e.g. "run `python b1_run.py -config b1.yaml` to regenerate Fig. 1"). Assignment explicitly requires this.

This project will implement a genetic algorithm (GA) for string search and memetic algorithm (MA) for traveling salesman problem. This section describes the experimental setup and design choices.

String search

A string search genetic algorithm was implemented and the used parameters are displayed in Table 1. "NaturalComputing" was used as the target string with a length L = 16, comprising of both lowercase and uppercase letters from an alphabet $|\Sigma|$. Since fitness is the number of characters that match the target, the maximum fitness is 16.

The algorithm aims to match the target string by evolving a population of N = 200 individuals. Each generation follows a few simple steps. First, during the tournament selection phase, a random subset K is drawn from the population N and the fittest string is kept for reproduction.

Larger K values mean more potentially high-fitness strings, which can lead to the selection of only the best performers, working as a filter against weaker candidates. While it usually leads to faster convergence, it can also reduce diversity as the entire population will mimic the strongest strings.

Then, each pair undergoes single-point crossover with a probability $p_c = 1$, producing 2 new offsprings. Crossover selects a random index and swaps the parent's characters beyond that cutoff. Additionally, every character of the offspring has a small mutation rate μ to randomly become one of the letters from the alphabet $|\Sigma|$. Finally, the entire parent generation is replaced with the new offspring generation.

All algorithms were run single-threaded on a desktop system with a AMD Ryzen 5800x processor.

Add one sentence explicitly stating the diversity measures used (Hamming mean pairwise distance on a subsample; position-wise Shannon entropy $H_i = -\sum_j f_{ij} \log f_{ij}$ averaged over positions), as required by the assignment appendix. Currently this is only visible via figure captions.

State the stopping criterion precisely: "A run is recorded as $t_{\text{finish}} = G_{\max}$ (right-censored) when the target is not reached." This matters for how the beeswarms should be read at $\mu = 0$ and $\mu = 3/L$.

Table 1: String Search GA Parameters

Parameter	**Value**
String length L	16
Alphabet $\|\Sigma\|$	52
Population size N	200
Tournament selection K	$\{2, 5\}$
Crossover probability p_c	1
Mutation rate μ	$\{0,\ 1/L,\ 3/L\}$ + sweep from 0/L to 10/L
Runs	10
Max generations $G_{\max}$	100

Table 2: Parameters for Exercise B1.

Caption of this table is fine. BUT: the report mentions $G_{\max} = 200$ in Fig. 2's caption while the parameter table says $G_{\max} = 100$. Pick one and make them consistent, or explicitly note that the cap differs between experiments and why.

Traveling Salesman Problem

Justify the chosen parameters ($N = 100$, $K = 5$, $p_c = 0.8$, $\mu = 0.3$): one sentence citing the lecture defaults or a quick pilot run. The rubric explicitly asks whether methods are justified.

Describe the custom 50-city instance (how generated — uniform random in a unit square? seed?) and which TSPLIB instance was used (eil51). Currently unclear — a reader cannot reproduce without the code.

For the TSP, both a simple evolutionary algorithm (EA) and a memetic algorithm (MA) were implemented. Each solution is encoded as a permutation of city indices representing a tour. The simple EA uses ordered crossover (OX): a random contiguous segment is copied from one parent and the remaining cities are filled in the order they appear in the second parent. Mutation is applied via swap mutation, where two randomly selected positions in the tour are exchanged. Tournament selection ($K = 5$) is used to select parents, and generational replacement with no elitism is applied.

The memetic algorithm extends the EA by applying 2-opt local search to each individual after mutation. The 2-opt procedure iteratively reverses sub-sequences of the tour to eliminate crossing edges, repeating until no improving swap can be found. This Lamarckian approach allows individuals to locally optimize their fitness before evaluation, which typically accelerates convergence. Both algorithms were evaluated on the custom 50-city dataset and on the TSPLIB benchmark instance *eil51*.

Add a row for the stopping criterion / generation budget and for the wall-clock budget used in Fig. 8/9. Also state the 2-opt termination rule ("until no improving 2-swap remains") already mentioned in prose — good, but make it a table row too for quick reference.

Results

Results currently consists of figures with captions doing most of the work. Rubric (section B "Description") penalises this. For each figure, add 2–3 sentences of narrative in the main text that (a) references the figure as "Fig. **??**", (b) states what was measured, (c) states the observation, (d) interprets it. This is the single highest-leverage change for the grade.

Table 3: String Search GA Parameters

FIX CAPTION: should be “TSP EA/MA Parameters”, not “String Search GA Parameters” — copy-paste error. Also fix dataset typo “eli51” → “eil51”.

Parameter	**Value**
Population size N	100
Tournament selection K	5
Crossover probability p_c	0.8
Mutation rate μ	0.3
Local search	2-opt
Runs	10
Datasets	eli51, TSPLIB, Custom 50-city instance

Table 4: Parameters for Exercise B2.

B1

In this section, we aim to answer whether and how the mutation rate μ trades convergence speed against diversity, and how the tournament size K impacts this. We first run the GA 10 times with the baseline values given by the exercise ($K = 2$, $\mu = 1/L$, $N = 200$) and then experiment with varying the values of μ and later K.

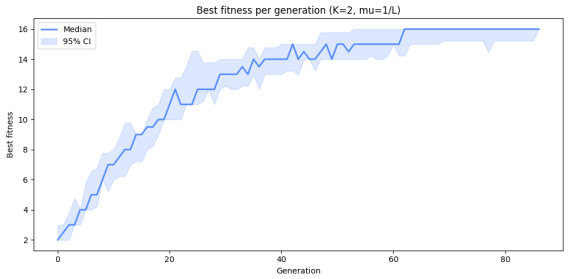

Figure 1: Results of running the GA 10 times with baseline values ($K = 2$, $\mu = 1/L$, $N = 200$). Blue line: median over 10 samples; shaded area: confidence interval.

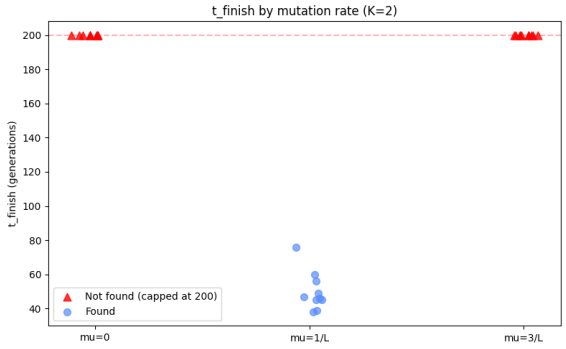

Figure 2: Three different mutation rates with $K = 2$. Only $\mu = 1/L$ finds the target before the 200-generation cutoff.

With the baseline values, the algorithm finds correct results within 86 generations for all 10 samples, but usually earlier (Figure 1). The average generation count till t_{finish} is 61.9. When changing μ to 0 or $3/L$, but keeping the other parameters equal, solutions are not found within 200 generations (Figure 2).

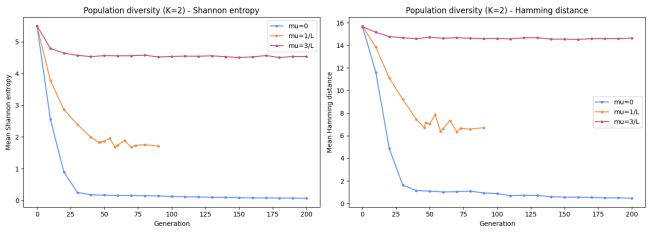

Figure 3: Measuring the diversity of runs in two different ways, with mean position-wise Shannon entropy and pairwise Hamming distance between strings over 10 runs.

$\mu = 0$	$\mu = 3/L$
`BaturalCrmputing` [14]	`NatusYOiHxpgJijg` [7]
`BaturalCrmputing` [14]	`UttKhaaYojputioh` [7]
`BaturalCSmputing` [14]	`iattIYvCojpgOTIg` [6]
`BaturalCrmputing` [14]	`iattIBYCojpgVbnk` [6]

Table 5: Top 4 individuals at generation 100 ($K = 2$, fitness in brackets). The $\mu = 1/L$ case is omitted because the population has already converged on the target.

Measuring the diversity of runs with different values of μ, both Shannon entropy and and Hamming distance show that mu=0 leads to low diversity, mu=3/L leads to very high diversity, while mu=1/L manages to strike a middle ground (and reach the target) (Figure 3). In the qualitative results, we can observe that $\mu = 0$ collapses to near-identical copies stuck at a local optimum; $\mu = 3/L$ stays far from the target `NaturalComputing` with very different strings (Table 5).

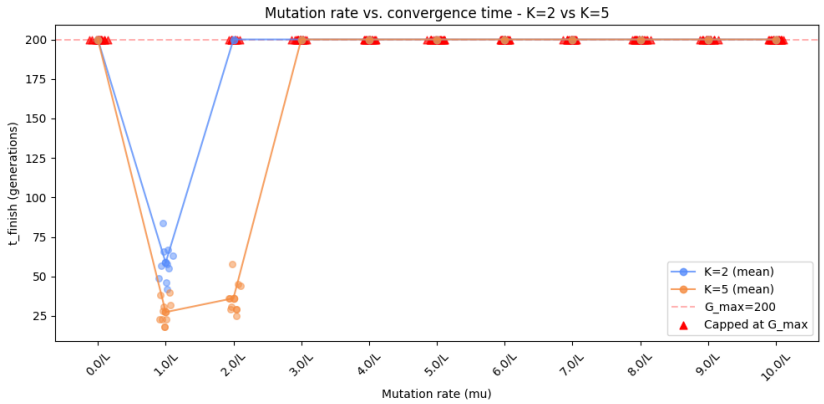

Figure 4: Joint graph exploring different μ for both $K = 2$ and $K = 5$. Runs are capped at 200 generations.

We repeat the mu sweep of Figure 2, varying K and μ more to see how a higher K interacts with mutation rate. K=5 leads to results faster and can also converge with mu=2/L, though mu=1/L remains optimal. As higher tournament size increases selection pressure, it can deal with more noise. For

B2

Add a short intro paragraph before the figures: restate the research question ("does 2-opt local search improve TSP performance beyond the extra compute it costs?"), name the two datasets, and preview that the fair-comparison answer requires wall-clock normalisation (you already argue this below — move the argument up so the figures land with context).

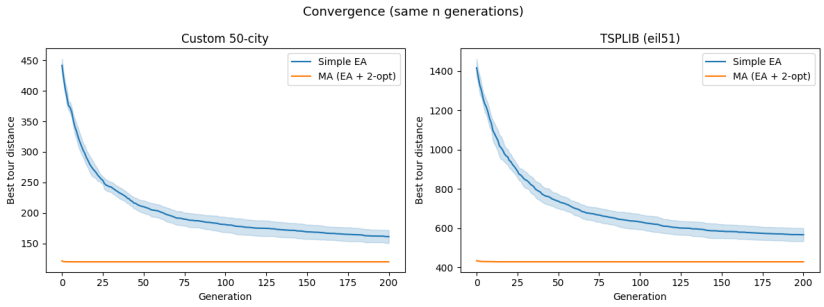

Figure 5: Both of the datasets show MA reaching better results in fewer generations, and in total.

Caption should name the datasets on each panel, the y-axis (mean best tour length in same units as the coordinates?), and what the shaded band is. Also state the gap numerically (e.g. MA $\approx X$, EA $\approx Y$ on eil51; known optimum for eil51 is 426, mention it).

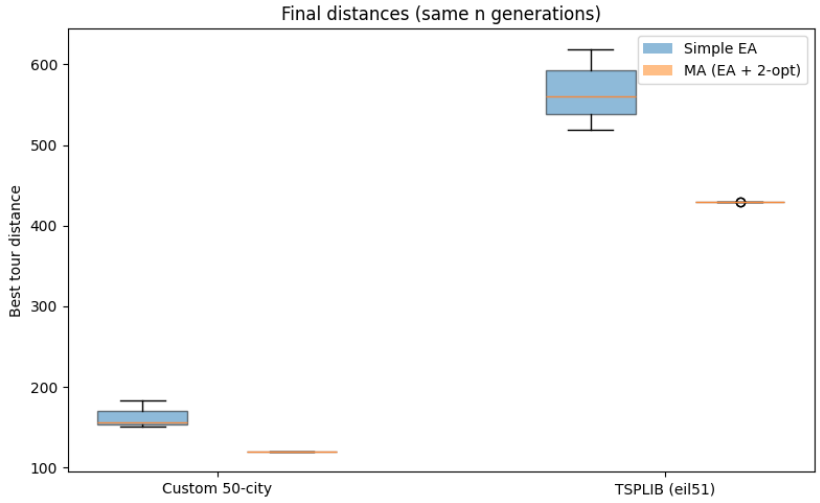

Figure 6: In this comparison, MA clearly beats EA in distance and consistency.

"Consistency" — quantify. Report std or IQR of final tour length across 10 runs for each (dataset, algorithm). Consider a Mann–Whitney U test on final tour lengths to support the claim with a p-value; the rubric ("Validity") rewards statistical backing of claims.

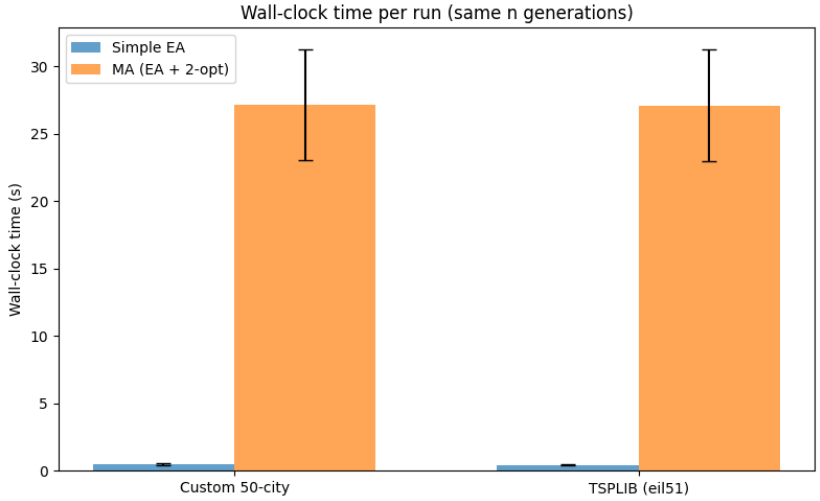

Figure 7: Per generation, MA takes much more compute & longer than EA. It is therefore not fair to compare them in this way.

We considered comparing them on number of operations, but the operations of MA are only in its neighborhood, so comparing them directly with whole generations of EA would again be unfair. We therefore decided to compare wall-clock time that, while influenceable by all kinds of factors and therefore not overly scientific, provides a good baseline for real-use performance, especially if averaged over 10 runs each.

Comparing the MA and simple EA on the same number of generations is not a fair baseline. Each MA generation includes a full 2-opt local search over every individual in the population, making it far more computationally expensive than a single EA generation. Any advantage observed per generation could therefore reflect additional computation rather than algorithmic superiority. We considered normalizing by number of fitness evaluations, but since 2-opt operates within a solution's neighborhood its evaluation count is not directly comparable to a full population generation in the EA. We therefore compare on wall-clock time, which - while sensitive to system-level factors - provides a practical performance baseline when averaged over 10 runs.

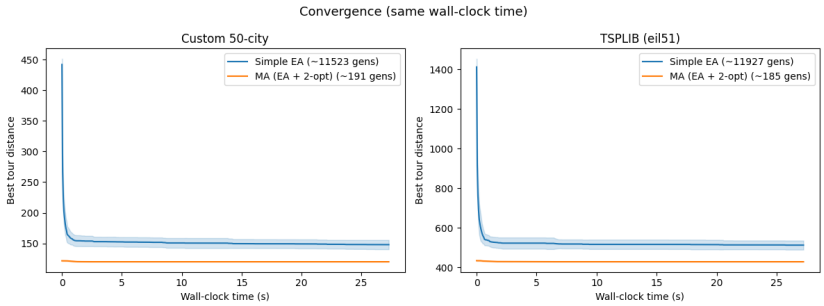

Figure 8: If compared instead on wall-clock time, MA still converges on better values, but EA's curve also flattens to near its best result in a comparable time as MA. While EA goes through more than 11-thousand generations in this time, MA only reaches around 190.

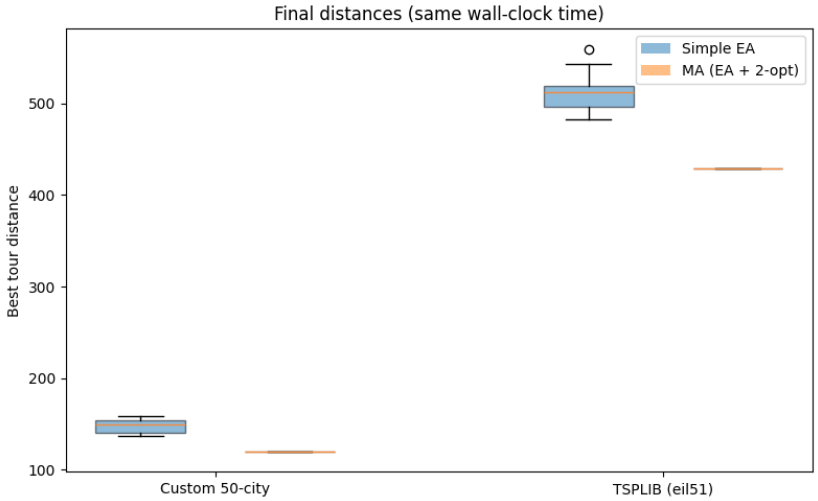

Figure 9: While the gap has shrunk considerably from comparing by number of generations, MA still provides better and more consistent results than EA.

Even under a fair wall-clock comparison, the MA consistently reaches shorter and more consistent tour distances than the simple EA on both datasets. The addition of local search therefore genuinely improves performance beyond what the extra computation alone can explain: 2-opt allows individuals to escape local optima that trap the plain EA, reducing both mean tour length and variance across runs.

This is the core conclusion of B.2 — strengthen it with numbers: "On eil51, MA reached mean tour length $X \pm s_X$ vs EA's $Y \pm s_Y$ within the same wall-clock budget of T s (Mann–Whitney U, $p < 0.01$)." Without numbers this reads as an assertion.

Note the two paragraphs above (the "We considered…" block and the "Comparing the MA…" block) are near-duplicates of each other — one is clearly an earlier draft. Keep the better one (the second is more polished) and delete the other.

Discussion

Several limitations of this study should be noted. All experiments were run on a single machine, and the results may differ on other hardware or with a more optimised 2-opt implementation, which are all factors which may affect the wall-clock time. With only 10 runs per condition the confidence intervals are modest, particularly at boundary mutation rates where success is inconsistent, and formal statistical tests (e.g. Wilcoxon rank-sum) would be needed to make stronger claims. Only two TSP instances were tested and only one target string was used for the string search task, so the generalisability

of the findings is limited. Regarding threats to validity, 2-opt is a heuristic specifically tailored to tour optimisation problems, so the advantage of the MA over the plain EA may not transfer to other combinatorial problems where no efficient local search is available. Additionally, censoring t_{finish} at $G_{\max} = 200$ when the target is not found biases mean convergence time downward, meaning the true performance gap between mutation rates is likely larger than reported. Future work could test the MA on larger TSPLIB instances to assess scalability, vary the intensity of local search (e.g. full 2-opt vs. a fixed number of improvement steps, or Baldwinian rather than Lamarckian learning), and explore adaptive mutation rates that adjust μ dynamically as population diversity changes.

Conclusion

This project shows that small changes in EA design choices have large and predictable effects on performance. For string search, the mutation rate $\mu = 1/L$ is optimal as it provides enough exploration to maintain diversity and avoid premature convergence, without being so disruptive that accumulated progress is lost. Increasing tournament size to $K = 5$ raises selection pressure, accelerating convergence and shifting the viable mutation range upward, though the optimum remains $\mu = 1/L$. For the TSP, the memetic algorithm consistently outperforms the plain EA in both solution quality and consistency, and this advantage holds even under a fair wall-clock comparison that accounts for the roughly 25 times higher cost per MA generation. Furthermore, 2-opt local search allows individuals to escape the local optima that cause high variability in the plain EA. The take-home message is that local search is worth its computational cost for TSP, while for string search there is a narrow optimal mutation rate whose precise value shifts with selection pressure. Both findings reflect the same underlying principle that effective evolutionary search requires a carefully tuned balance between exploration and exploitation.

Appendix

Part A

A1

What can you conclude about the effects of fitness scaling on selection pressure?

Adding constants raises all fitness all values by that amount. So it smooths out all the values. For example, when we compare f2 (which only squares X) and f4 (which squares X and adds 20), we can see that the probabilities of f4 are closer together. Probabilities are increased and reduced, so there is a much smaller gap between them, which also reduces selection pressure. Multiplying by a constant does not change the selection probabilities. If we compare f2 and f3, we can see that the probabilities stay the same even if we scale it. So scaling fitness does not have an effect on the selection pressure.

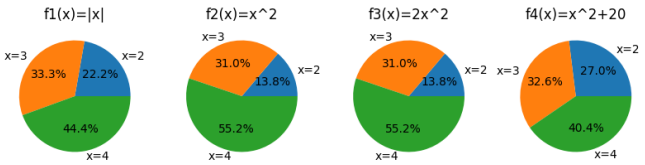

Figure 10: Selection probability of the three individuals for each function.

x	f1	p1	f2	p2	f3	p3	f4	p4
2	2	0.2222	4	0.1379	8	0.1379	24	0.2697
3	3	0.3333	9	0.3103	18	0.3103	29	0.3258
4	4	0.4444	16	0.5517	32	0.5517	36	0.4045

Table 6: Fitness function value and selection probability for each individual and each function.

A2

Does the algorithm find the optimum? Yes. It is able to reach fitness 100, meaning all 100 bits were successfully set to 1.

What do you see? The version without selection is stuck at around 50 and never converges. The version with selection keeps steadily climbing, ultimately reaching 100.

Can you conclude anything on the difference between these two versions based on the above results? If yes, why? If not, what should you do to make a fair comparison? No. We would need multiple runs as EAs are stochastic and each run produces different results because of initialization and random mutation. To make it fair, we should run each version multiple times and use averages.

Is there a difference in performance? Yes. It is apparent that there is a difference in performance between the versions. Across all the runs, the version with selection always converges to fitness 100 while the version without selection is always near 50 through all the 1500 generations. The algorithm has no memory of good solutions and cannot improve.

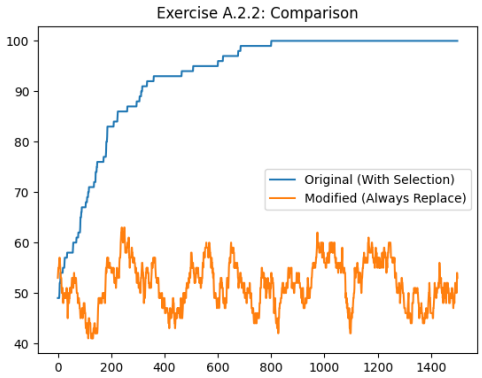

Figure 11: Fitness against generations.

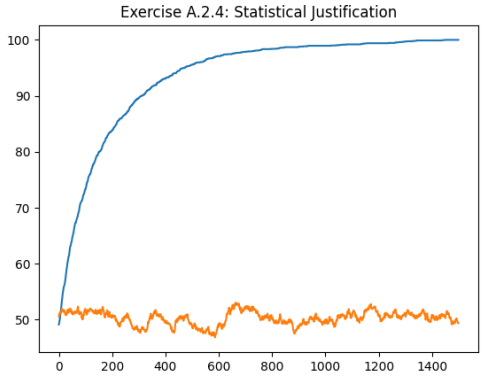

Figure 12: Averages of fitness vs generations across multiple runs.